PRIVATE KNOWLEDGE BASE

A Local-First Retrieval-Augmented Generation (RAG) Application

PROJECT VISION

Build a practical question-answering system that lets users upload private notes and PDF documents, ask questions in natural language, and receive answers grounded in retrieved passages—with source filenames and page numbers. The first version runs on your own computer, avoiding paid LLM APIs and cloud GPU hosting.

THE CORE IDEA

This is more than a chatbot. It is an end-to-end AI application demonstrating document processing, semantic search, vector databases, local LLM inference, backend API design, and a React interface.

Build locally first. Make the pipeline reliable. Add evaluation and security before considering deployment.

PROJECT GOALS

- Process PDF notes and later DOCX and TXT files.
- Split extracted text with RecursiveCharacterTextSplitter.
- Generate local embeddings with Sentence Transformers.
- Store vectors and metadata in ChromaDB.
- Retrieve relevant and diverse passages with Maximal Marginal Relevance (MMR).
- Generate grounded answers with an Ollama model running locally.
- Expose the workflow through FastAPI and Pydantic, with a React frontend.
- Support persistent indexing, source citations, and document management.

FIRST MILESTONE: Upload one text-based PDF, ask a question, and show the answer with its source page.

1. System architecture

Keep document ingestion separate from question answering so files do not need to be re-embedded for every question.

Stage	Technology / responsibility
User interface	React: upload documents, view indexing status, submit questions, display answers and sources.
API	FastAPI + Pydantic: validate requests, return structured responses, and handle errors.
Ingestion	LangChain document loaders such as PyPDFLoader extract text and metadata.
Chunking	RecursiveCharacterTextSplitter creates chunks with overlap while preserving metadata.
Embeddings	Sentence Transformers embeds each chunk locally and embeds each user query with the same model.
Vector store	ChromaDB persists vectors, chunk text, filenames, page numbers, and document IDs.
Retrieval	MMR retrieves relevant and diverse chunks to pass to the prompt.
Generation	Ollama runs a locally downloaded model to answer using the question and retrieved context.

Recommended local model setup

- LLM: Start with a small Ollama model such as Qwen2.5 3B; benchmark speed and answer quality on your own machine.
- Embeddings: Start with sentence-transformers/all-MiniLM-L6-v2. Compare BAAI/bge-small-en-v1.5 using your own retrieval tests.
- Hardware: Model file size is not runtime memory. RAM/VRAM use depends on quantization, context length, and concurrent requests.
- Offline: Download models first, configure local paths if needed, and test with the internet disconnected.

Ollama runs the LLM; Sentence Transformers creates embeddings. These are separate parts of the system.

2. Application workflow

Document ingestion — performed when a document is added or re-indexed

- Validate the uploaded file extension, size, and content.
- Load the document with an appropriate LangChain loader.
- Split extracted text and retain filename, page number, document ID, and chunk ID.
- Generate embeddings locally and add chunk records to ChromaDB.
- Report indexing status and extraction errors to the frontend.

Question answering — performed for each query

- Validate the question with a Pydantic request model.
- Embed the query using the same embedding model used for indexing.
- Retrieve candidate chunks from ChromaDB using MMR.
- Build a prompt that clearly separates retrieved context from the user question.
- Ask Ollama to answer from the supplied context and admit when evidence is insufficient.
- Return the answer, source filenames, page numbers, and optionally relevant excerpts.

Suggested API endpoints

Method / endpoint	Purpose
`POST /documents/upload`	Upload and index a document.
`GET /documents`	List indexed documents and status.
`DELETE /documents/{id}`	Remove a document and its vectors.
`POST /chat`	Retrieve context and generate a sourced answer.
`DELETE /knowledge-base`	Clear indexed data after an explicit, authorized action.
`GET /health`	Report API and model readiness.

No conversation memory is required for version 1. Each request can be stateless: retrieve document context, answer, and return sources. Add chat history only if it becomes a deliberate requirement.

3. Design choices that make the project stronger

Persist ChromaDB instead of deleting it at shutdown

Use a persistent local directory for the ChromaDB collection. On restart, reopen the collection and reuse existing embeddings. Provide an explicit “Clear knowledge base” action. Automatic cleanup is appropriate only for a deliberately temporary workspace.

Cite sources in the answer

Store filename, page number, document ID, and chunk ID in metadata. Return citations with the answer so users can verify claims against original notes. Preserve page metadata during splitting and test it with multi-page PDFs.

Handle PDF limitations

Text-based PDFs are the easiest starting point. Scanned pages may require OCR; tables, equations, diagrams, and unusual layouts may need specialized parsing. Make empty or failed extraction visible rather than silently indexing empty content.

Use a grounded prompt

Tell the model to treat retrieved text as reference material, not instructions; answer only when the context supports it; state when information is missing; and cite supplied source labels. Prompting reduces risk but does not eliminate hallucinations or prompt injection.

Treat uploaded documents as untrusted input

Validate uploads, set file-size limits, avoid unsafe file paths, and do not expose Ollama or ChromaDB directly to the public internet. For multiple users, add authentication and enforce document-level permissions during retrieval.

Measure quality rather than assuming it works

Create an evaluation set of real questions with expected source pages and acceptable answers. Compare chunk size, overlap, embedding models, MMR settings, and retrieval count. Measure retrieval quality separately from answer correctness.

4. Build roadmap

Phase 1 — CLI prototype

Load one PDF, chunk it, embed it, store it in ChromaDB, retrieve with MMR, and answer through Ollama.

Phase 2 — Reliable ingestion

Support multiple PDFs, metadata, duplicate handling, document deletion, and persistent indexing.

Phase 3 — FastAPI

Create Pydantic schemas, upload/chat/document endpoints, clear errors, and health checks.

Phase 4 — React interface

Build upload progress, indexing status, chat UI, citations, and document management.

Phase 5 — Evaluation

Test retrieval and answer quality on a fixed set of questions and document known limitations.

Phase 6 — Portfolio polish

Add README, architecture diagram, setup instructions, screenshots, a short demo video, and sample non-sensitive documents.

5. Resume-ready description

Private Knowledge Base — Local RAG Application

- Developed an end-to-end Retrieval-Augmented Generation (RAG) application using LangChain, FastAPI, React, ChromaDB, and Ollama.
- Implemented PDF ingestion, recursive text chunking, and local Sentence Transformers embeddings for semantic search.
- Integrated Maximal Marginal Relevance (MMR) retrieval to find relevant and diverse document passages.
- Built a source-grounded question-answering pipeline using a locally hosted LLM, with document and page-level citations.
- Designed REST APIs with FastAPI and Pydantic for document uploads, indexing, retrieval, and question answering.

Only list features you have actually implemented and tested. Add measured results—such as retrieval accuracy, response latency, and supported document count—only after benchmarking.

6. What to demonstrate on GitHub

- A clear README with architecture, setup steps, dependencies, and model-download instructions.
- An architecture diagram and screenshots of upload, chat, and source citations.
- A short screen recording showing a question, retrieved sources, and the final answer.
- A small, non-sensitive sample knowledge base and evaluation question set.
- A privacy note explaining which components run locally and what data, if any, leaves the machine.

FINAL PRINCIPLE

Build locally first. Make it reliable before making it bigger. The value of this project is the complete working pipeline, source-grounded answers, thoughtful evaluation, and clear engineering decisions—not expensive cloud infrastructure.

Suggested stack: Python · LangChain · PyPDFLoader · RecursiveCharacterTextSplitter · Sentence Transformers · ChromaDB · Ollama · FastAPI · Pydantic · React